

Defining Functions

In Python, a function is a group of related statements that performs a specific task. Functions help break our program into smaller and modular chunks. Furthermore, it avoids repetition and makes the code reusable.

Syntax of Function

```
def function_name(parameters):  
    """docstring"""  
    statement(s)
```

- The keyword `def` introduces a function definition. It must be followed by the function name and the parenthesized list of formal parameters. The statements that form the body of the function start at the next line, and must be indented.
- The first statement of the function body can optionally be a string literal; this string literal is the function's documentation string, or docstring. It's good practice to include docstrings in code that you write, so make a habit of it.

Example of a function definition

In [10]:

```
def fun(msg):  
    '''  
    Prints welcome message  
    and username  
    '''  
    print("welcome ",msg)
```

How to call a function in python?

In [11]:

```
fun("Khyati")
```

welcome Khyati

We can create a function that writes the Fibonacci series to an arbitrary boundary

In [2]:

```
def fib(n): # write Fibonacci series up to n
    """Print a Fibonacci series up to n."""
    a, b = 0, 1
    while a < n:
        print(a,end=" ")
        a, b = b, a+b

fib(20)
```

0 1 1 2 3 5 8 13

Docstrings

The first string after the function header is called the docstring and is short for documentation string. It is briefly used to explain what a function does. Although optional, documentation is a good programming practice. This string is available to us as the `__doc__` attribute of the function.

One can write `print(fib.__doc__)` or `?fib` to get docstring.

In [3]:

```
print(fib.__doc__)
```

Print a Fibonacci series up to n.

In [4]:

```
?fib
```

The return statement

Syntax of return

- `return [expression_list]`

In [36]:

```
def absolute_value(num):
    """This function returns the absolute
    value of the entered number"""

    if num >= 0:
        return num
    else:
        return -num
```

In [37]:

```
print(absolute_value(2))  
print(absolute_value(-4))
```

2
4

Function Arguments

- Default arguments
- Keyword arguments
- Variable-length arguments

Default Argument

A default argument is an argument that assumes a default value if a value is not provided in the function call for that argument.

We can provide a default value to an argument by using the assignment operator (=).

In [42]:

```
def greet(name, msg="Good morning!"):
    """
    This function greets to
    the person with the
    provided message.
    """
    print("Hello", name + ', ' + msg)
```

In [45]:

```
greet("Khyati")  
greet("Khyati", "How do you do?") #default value will be overwritten
```

Hello Khyati, Good morning!
Hello Khyati, How do you do?

In this function, the parameter name is mandatory. msg has a default value of "Good morning!". So, it is optional during a call. If a value is provided, it will overwrite the default value.

Any number of arguments in a function can have a default value. But once we have a default argument, all the arguments to its right must also have default values.

This means to say, non-default arguments cannot follow default arguments.

Example:

```
def greet(msg = "Good morning!", name):
```

We would get an error as:
SyntaxError: non-default argument follows default argument

Keyword Argument

When we call a function with some values, these values get assigned to the arguments according to their position. Python allows functions to be called using keyword arguments. When we call functions in this way, the order (position) of the arguments can be changed.

In [6]:

```
def greet(name, msg):  
    """  
    This function greets to  
    the person with the  
    provided message.  
    """  
    print("welcome", name + ', ' + msg)
```

In [7]:

```
greet("Khyati", "hello")  
greet(msg="hello", name="Khyati")  
greet("Khyati", msg="hello")  
#greet(msg="hello", "Khyati") positional argument can't follow keyword argument
```

```
welcome Khyati, hello  
welcome Khyati, hello  
welcome Khyati, hello
```

Try it yourself

1. Write a function program to print fibonacci series.

Python Arbitrary Arguments

Sometimes, we do not know in advance the number of arguments that will be passed into a function. Python allows us to handle this kind of situation through function calls with an arbitrary number of arguments. In the function definition, we use an asterisk (*) before the parameter name to denote this kind of argument. Here is an example.

In [2]:

```
def greet(*names):  
    """This function greets all  
    the person in the names tuple."""  
  
    # names is a tuple with arguments  
    for name in names:  
        print("Hello", name)  
  
greet("Aarav", "Antra", "Reena")  
greet("Aarav", "Antra")  
greet("Aarav")
```

Hello Aarav
Hello Antra
Hello Reena
Hello Aarav
Hello Antra
Hello Aarav

Recursion in python

In [1]:

```
def factorial(x):  
    """This is a recursive function  
    to find the factorial of an integer"""  
  
    if x == 1:  
        return 1  
    else:  
        return (x * factorial(x-1))  
factorial(4)
```

Out[1]:

24

Try it yourself

Find sum of elements of an array.